FACULTY OF COMPUTING TECHNOLOGY

DEPARTMENT OF COMPUTER SCIENCE

GPS Navigation System

Project Report: GPS Navigation System using Dijkstra's Algorithm

Course:	Algorithm Design & Analysis
Instructor:	*EQUBAY*
Submission Date:	_____

GROUP MEMBERS

Haftom G/hiwet	1601840
Daniel Gidey	1601688
Awetash Sbhatu	1602233

Table of Contents

1. Introduction – Real-World Problem Context 1

2. Algorithm Design 2

2.1 Dijkstra's Algorithm 2

2.2 Why Dijkstra's Algorithm is Used 2

2.3 Dijkstra's Algorithm – Pseudocode 3

2.4 Relaxation Step (Concept) 3

3. Implementation Details 4

3.1 Adjacency Matrix 4

3.2 Key Features 4

4. Complexity Analysis 5

4.1 Comparison with Optimized Approach 5

4.2 Array-Based Implementation (Used in This Project) 6

5. Experimentation 7

5.1 Dataset Description 7

5.2 Test Cases 7

5.3 Resulting Shortest Paths (Output) 8

5.4 Relaxation Steps (Example) 10

6. Challenges & Solutions 11

7. Conclusion 12

8. References 13

1. Introduction - Real-World Problem Context

Modern navigation systems such as GPS devices and map applications must determine the shortest path between two locations in a road network. Applications like Google Maps, Waze, and other navigation tools solve this problem by modeling the road network as a graph where intersections or cities are represented as vertices and roads connecting them are represented as edges with distances as weights.

> **Finding the shortest route between locations is a fundamental problem in computer science known as the Shortest Path Problem. Efficient solutions are essential because road networks can contain thousands or even millions of nodes.**

One of the most widely used algorithms for solving this problem when edge weights are non-negative is Dijkstra's Algorithm. It computes the shortest path from a source vertex to all other vertices in the graph.

In this project, we implement a simplified GPS navigation system that calculates the shortest path between cities using Dijkstra's Algorithm. The road network is represented using an adjacency matrix and the algorithm uses arrays to track distances, visited vertices, and parent relationships to reconstruct the optimal path.

The system reads road data from a CSV dataset, constructs the graph, and computes the shortest path between selected cities. The algorithm also demonstrates the relaxation process and outputs the final path and distance.2. Algorithm Design

2.1 Dijkstra's Algorithm

> **Dijkstra's Algorithm is a greedy shortest path algorithm used to compute the minimum distance from a source vertex to all other vertices in a weighted graph with non-negative edge weights. The algorithm maintains a set of visited vertices and repeatedly selects the vertex with the smallest known distance. It then updates (relaxes) the distances of its neighboring vertices if a shorter path is found.**

The key idea is that once a vertex is marked as visited, its shortest distance is finalized and cannot be improved further. This guarantees that the algorithm always produces the optimal shortest path.

2.2 Why Dijkstra's Algorithm is Used

Dijkstra's Algorithm is used because it is specifically designed to find the shortest path in a graph with non-negative edge weights, which matches the conditions of a road network.

In a GPS Navigation System:

- Cities are represented as vertices
- Roads are represented as edges
- Distances between cities are positive values (km)

Since all distances are non-negative, Dijkstra's Algorithm guarantees the correct and optimal shortest path from the source city to all other cities.

> **Additionally, the algorithm is:**

Dijkstra's Algorithm - Pseudocode

The following pseudocode describes the process used to compute the shortest path from a source city to all other cities in the graph.

```
DIJKSTRA(Graph, source)

Input:
    Graph represented as adjacency matrix
    source = starting city

Output:
    dist[]   → shortest distance from source to each city
    parent[] → previous city in the shortest path

1. Let V be the number of cities

2. For each city v from 0 to V-1
        dist[v] = ∞
        visited[v] = false
        parent[v] = -1
```

```
3. dist[source] = 0

4. Repeat V times

      u = vertex with minimum dist[u] among all unvisited vertices

      visited[u] = true

      For each city v from 0 to V-1

          if graph[u][v] > 0 AND visited[v] = false then

              if dist[u] + graph[u][v] < dist[v] then

                      dist[v] = dist[u] + graph[u][v]

                      parent[v] = u

5. End

Return dist[] and parent[] make it structured
```

RELAXATION STEP (CONCEPT)

Relaxation is the process of updating the shortest distance when a better path is found.

```
RELAX(u, v, weight)
 if dist[u] + weight < dist[v] then
   dist[v] = dist[u] + weight
    parent[v] = u
```

3. Implementation Details

3.1 Adjacency Matrix

Graph Sample (First 5 Cities)

```
      0   1   2   3   4   5   6
    -------------------------------
 0 |  0  10  15  25   0   8   0
 1 | 10   0   0  12  30   0   0
 2 | 15   0   0   0  10  22   0
 3 | 25  12   0   0   2   0  28
 4 |  0  30  10   2   0   5   0
```

Advantages:

✓ O(1) edge lookup

✓ Simple implementation

✓ Efficient for dense graphs

Disadvantages:

✗ $O(V^2)$ space usage

✗ Inefficient for sparse graphs

✗ Memory overhead for large graphs

3.2 Key Features

1. Undirected Graph

- Each road is bidirectional

- Edge (u,v) = Edge (v,u)

2. Weighted Edges

- Distance in kilometers

- Positive weights only

3. Multi-Query Support

- Find multiple paths in one session

- Scanner remains open for efficiency

- Results append to file

4. Step-by-Step Visualization

- Shows algorithm progress

- Displays visited vertices

- Shows distance updates

4. Complexity Analysis

4.1 Comparison with Optimized Approach

> **The efficiency of Dijkstra's Algorithm is fundamentally determined by the data structure used to manage vertex selection and edge relaxation. The difference between implementations is not the algorithm itself, but how efficiently it performs its core operations.**

A. OPTIMIZED IMPLEMENTATION (PRIORITY QUEUE / MIN-HEAP)

The optimized implementation of Dijkstra's Algorithm improves efficiency by using a priority queue (min-heap) to manage vertices based on their shortest distance. Instead of scanning all vertices, it always selects the minimum-distance vertex efficiently.

Core Data Structures

- Adjacency List - Stores only existing edges, making it memory-efficient for large graphs.
- Min-Heap (Priority Queue) - Stores pairs (distance, vertex) and always returns the vertex with the smallest distance.
- dist[] (Distance Array) - Stores the shortest known distances from the source.
- parent[] (Path Array) - Used to reconstruct the shortest path.

Key Operations

- Extract-Min Operation: Removes the smallest distance vertex from the heap. Time = O(log V)

- Relaxation with Heap Update: Updating and inserting into heap. Time = O(log V) per update

Time Complexity = O((V+E) log V)

Advantages:

✓ Highly efficient for large-scale graphs

✓ Suitable for sparse graphs

✓ Reduces unnecessary scanning of vertices

✓ Scales well for real-world navigation systems

Disadvantages:

✗ More complex to implement

✗ Requires additional data structures (heap, adjacency list)

✗ Higher implementation overhead for beginners

B. ARRAY-BASED IMPLEMENTATION (USED IN THIS PROJECT)

The array-based implementation of Dijkstra's Algorithm uses simple arrays and an adjacency matrix to compute the shortest path between vertices. It relies on sequential scanning to select the next vertex and update distances.

Core Data Structures

- Adjacency Matrix (V x V) - Stores the graph where each cell graph[u][v] represents the distance between cities. If no edge exists, the value is 0 or infinity.
- dist[] (Distance Array) - Stores the shortest known distance from the source to each vertex.
- visited[] (Boolean Array) - Keeps track of vertices whose shortest distance is already finalized.
- parent[] (Path Array) - Stores the previous vertex to reconstruct the shortest path.

Time Complexity: $O(V^2)$

Dijkstra's Algorithm (array-based implementation) runs in quadratic time due to repeated scanning of all vertices.

- The algorithm executes V iterations (one for each city)
- In each iteration: It finds the minimum distance vertex -> O(V); It checks all neighboring vertices -> O(V)

Total Time = O(V x V) = $O(V^2)$

Number of cities (V) = 100 | Approximate operations = 10,000

Advantages:

✓ Simple and easy to implement

✓ No need for complex data structures

✓ Efficient for small to medium-sized graphs

✓ Suitable for dense graphs (many edges)

✓ Easy to debug and visualize (important for le

Disadvantages:

✗ Higher time complexity compared to optimized version

✗ Inefficient for very large graphs

✗ Requires $O(V^2)$ memory for adjacency matrix

✗ Performs unnecessary scans of vertices

The array-based approach was chosen for this project for the following reasons:

-The problem size is relatively small (100 cities), making (O(V^2)) acceptable

-The project requirement emphasizes simplicity and clarity of implementation

-It allows easy understanding of Dijkstra's algorithm step-by-step

-The adjacency matrix simplifies graph representation and edge lookup

-It is ideal for demonstrating algorithm concepts in an academic setting

SPACE COMPLEXITY: $O(V^2)$

The space complexity is dominated by the adjacency matrix used to store the graph.

Component	Space Requirement
Adjacency Matrix	$O(V^2)$
Distance array dist[]	O(V)
Visited array visited[]	O(V)
Parent array parent[]	O(V)
Total	**$O(V^2) + 3xO(V) \approx O(V^2)$**

5. Experimentation

5.1 Dataset Description

The system was tested using a generated dataset representing a road network:

Total Cities (Vertices): 100

Total Roads (Edges): 500+

Distance Range: 3 km – 119 km

Graph Type: Undirected weighted graph

Storage: CSV file (source, destination, distance)

5.2 Test Cases

Test Case	Source	Destination	Expected Result	Actual Result	Status
TC1	5	7	Path exists	5 -> 6 -> 7 (20 km)	✓ Pass
TC2	0	4	Shortest path	0 -> 5 -> 4 (13 km)	✓ Pass
TC3	2	3	Path exists	2 -> 4 -> 3	✓ Pass
TC4	10	99	Reachable	Correct distance	✓ Pass
TC5	8	50	Reachable	Correct path	✓ Pass

```
                GPS Navigation System (Dijkstra)
   ==========================================================

     Enter Source City      (0 - 99):
→  0

     Enter Destination City (0 - 99):
→  12


   ------------------------------------------------------------
   ------------------------------------------------------------
     Dijkstra Algorithm Steps
   ------------------------------------------------------------

   Step 1
     Visited City : 0
     Visited Cities : {0}
     Relax        : 0->1=10   0->2=15   0->3=25   0->5=8   0->10=25   0->22=87   0->51=23   0->90=30   0->97=40   0->99=
     dist[]       : [ 0 10 15 25 INF 8 INF ... ]

   Step 2
     Visited City : 5
     Visited Cities : {0, 5}
     Relax        : 5->4=13   5->6=19   5->8=26   5->11=97   5->15=28   5->35=112   5->42=115   5->95=36   5->98=100
     dist[]       : [ 0 10 15 25 13 8 19 ... ]

   Step 3
     Visited City : 1
     Visited Cities : {0, 1, 5}
Step 4
  Visited City : 4
  Visited Cities : {0, 1, 4, 5}
  Relax        : 4->3=15   4->7=48   4->12=45   4->14=36   4->16=101   4->43=53   4->73=30   4->85=59   4->94=38
  dist[]       : [ 0 10 15 15 13 8 19 ... ]

Step 5
  Visited City : 2
  Visited Cities : {0, 1, 2, 4, 5}
  Relax        : 2->12=37   2->32=47   2->34=48   2->92=42
  dist[]       : [ 0 10 15 15 13 8 19 ... ]

Step 6
  Visited City : 3
  Visited Cities : {0, 1, 2, 3, 4, 5}
  Relax        : 3->13=37   3->59=28   3->93=42
  dist[]       : [ 0 10 15 15 13 8 19 ... ]

Step 7
  Visited City : 6
  Visited Cities : {0, 1, 2, 3, 4, 5, 6}
  Relax        : 6->7=28   6->9=59   6->16=39   6->48=45   6->83=106   6->96=47
  dist[]       : [ 0 10 15 15 13 8 19 ... ]

------------------------------------------------------------
  Shortest Path Result
------------------------------------------------------------
  Source City      : 0
  Destination City : 12
------------------------------------------------------------
```

```
------------------------------------------------------------
Shortest Path Result
------------------------------------------------------------
Source City      : 0
Destination City : 12
------------------------------------------------------------
Shortest Distance: 37 km

Path: 0 -> 2 -> 12

------------------------------------------------------------
Shortest Distances from City 0 (5 samples)
------------------------------------------------------------
City 10 : 25 km
City 25 : 45 km
City 50 : 30 km
City 75 : 46 km
City 99 : 20 km

============================================================
             Navigation Calculation Completed
============================================================
```

5.4 Relaxation Steps (Example)

Source City = 5 | Destination City = 7

1 Visited: 5

Relax: 4 → 3 = 7, 4 → 2 = 15

Updated distances: dist[3]=7, dist[2]=15

2 Visited: 4Relax:

Relax: 4 → 3 = 7, 4 → 2 = 15

Updated distances: dist[3]=7, dist[2]=15

3 Visited: 3

Relax: 3 → 1 = 19

Updated distances: dist[1]=19

4 Visited: 0

Relax: 0 → 1 = 18 (better path)

Updated distances: dist[1]=18

5 **Visited: 6** **Relax:** 6 → 7 = 20

Shortest Path: 5 -> 6 -> 7 = 20 km

6. Challenges & Solutions

Challenge	Solution
Duplicate edges in roads.csv causing incorrect graph representation	Implemented edge validation and counting logic to detect and prevent duplicate edges during CSV loading.
Missing road connections (399 edges instead of expected 501)	Cleaned the dataset and added 101 verified unique edges to complete the road network.
Scanner object closing during recursive method calls	Declared a single global Scanner instance and reused it across the entire application.
File output being overwritten each time the program runs	Enabled append mode in file writing to preserve previous results.
Shortest path reconstruction difficulty after running Dijkstra's	Stored parent nodes in a parent[] array to trace and reconstruct the final shortest path.

7. Conclusion

The GPS Navigation System project successfully demonstrates the application of Dijkstra's Shortest Path Algorithm for finding optimal routes in a road network. The system models a transportation network consisting of 100 cities connected by 501 roads, where cities are represented as vertices and roads are represented as weighted edges in an undirected graph.

By representing the graph using an adjacency matrix, the system efficiently stores and processes the distance information between cities. The implementation of Dijkstra's algorithm computes the shortest distance from a selected source city to all other cities while maintaining the actual path using the parent array. This allows the program not only to calculate the minimum distance but also to reconstruct the sequence of cities along the optimal route.

The algorithm demonstrates $O(V^2)$ time complexity when implemented with arrays, which satisfies the project requirement. Through step-by-step relaxation and vertex selection, the algorithm gradually updates the shortest known distances until the optimal paths are determined.

The project also includes features such as user input for selecting source and destination cities, clear output formatting, and result storage, making the system practical and easy to use. Additionally, the use of a CSV dataset with more than 500 road connections provides a realistic simulation of a navigation network.

Overall, this project provided valuable experience in graph data structures, algorithm implementation, and performance analysis. It highlights how fundamental algorithms such as Dijkstra's algorithm are used in real-world applications such as GPS navigation, transportation planning, and network routing systems.

Key Achievements

- Correct implementation of **Dijkstra's shortest path algorithm**
- Efficient **$O(V^2)$ array-based solution** as required by the assignment
- Graph representation using an **adjacency matrix**
- Implementation of **distance, visited, and parent arrays**
- Step-by-step **relaxation demonstration for small graphs**
- Ability to compute shortest paths for **large datasets (100 cities, 500+ roads)**
- Clear output showing **distance and path reconstruction**
- Well-structured code and **comprehensive documentation**

8. References

1. Cormen, T. H., et al. Introduction to Algorithms, MIT Press

2. Dijkstra, E. W. (1959). A Note on Two Problems in Graphs

3 GeeksforGeeks – Dijkstra's Algorithm

4. Generated Dataset (Road Network Data)